

SKILL-04

TASK-1 : Process Synchronization and Child Process Monitoring Using waitpid()

```
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid1, pid2;

    pid1 = fork();

    if (pid1 == 0) {
        printf("Child 1 started. PID = %d\n", getpid());
        sleep(3);
        printf("Child 1 completed.\n");
        exit(0);
    }

    pid2 = fork();

    if (pid2 == 0) {
        printf("Child 2 started. PID = %d\n", getpid());
        sleep(5);
        printf("Child 2 completed.\n");
        exit(0);
    }

    printf("Parent Process. PID = %d\n", getpid());

    waitpid(pid1, NULL, 0);
    printf("Parent detected Child 1 completion.\n");

    waitpid(pid2, NULL, 0);
    printf("Parent detected Child 2 completion.\n");

    printf("All child processes completed.\n");

    return 0;
}
```

Result:

```
Parent Process. PID = 766
Child 2 started. PID = 768
Child 1 started. PID = 767
Child 1 completed.
Parent detected Child 1 completion.
Child 2 completed.
Parent detected Child 2 completion.
All child processes completed.
root@SRIVALLI:~# |
```

Report:

In this task, a parent process created multiple child processes using the `fork()` system call. The `waitpid()` function was used to monitor and wait for specific child processes to complete their execution. This helped in synchronizing the parent and child processes and ensured that the parent process continued only after the child processes finished. The execution order and process completion were observed successfully. Multiple child processes were created successfully, and the parent process monitored their completion using `waitpid()`. Process synchronization was achieved successfully.

Task-2: To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution

```
#include <unistd.h>

int main() {
    char *path = getenv("PATH");

    if (path == NULL) {
        printf("PATH variable not found.\n");
        return 1;
    }

    printf("PATH Variable:\n%s\n\n", path);

    char temp[1024];
    strcpy(temp, path);

    printf("Directories in PATH:\n");

    char *dir = strtok(temp, ":");

    while (dir != NULL) {
        printf("%s\n", dir);
        dir = strtok(NULL, ":");
    }

    char command[100];

    printf("\nEnter command name: ");
    scanf("%s", command);

    char *path_copy = getenv("PATH");
    strcpy(temp, path_copy);

    dir = strtok(temp, ":");

    while (dir != NULL) {
        char fullpath[1024];
```

```

while (dir != NULL) {
    printf("%s\n", dir);
    dir = strtok(NULL, ":");
}

char command[100];

printf("\nEnter command name: ");
scanf("%s", command);

char *path_copy = getenv("PATH");
strcpy(temp, path_copy);

dir = strtok(temp, ":");

while (dir != NULL) {
    char fullpath[1024];

    sprintf(fullpath, "%s/%s", dir, command);

    if (access(fullpath, X_OK) == 0) {
        printf("\nExecutable Found:\n%s\n", fullpath);
        return 0;
    }

    dir = strtok(NULL, ":");
}

printf("\nCommand not found in PATH.\n");

return 0;

```

Result:

```

/mnt/c/WINDOWS/System32/WindowsPowerShell/v1.0/
/mnt/c/WINDOWS/System32/OpenSSH/
/mnt/c/Program Files/dotnet/
/mnt/c/Program Files/nodejs/
/mnt/c/Users/sriva_6wg39hp/AppData/Local/Microsoft/WindowsApps
/mnt/c/Users/sriva_6wg39hp/AppData/Local/Programs/Microsoft V
/mnt/c/Program Files/JetBrains/PyCharm 2026.1/bin
/mnt/c/Users/sriva_6wg39hp/AppData/Roaming/npm
/mnt/c/Users/sriva_6wg39hp/AppData/Local/Kingsoft/WPS Office/
itool
/snap/bin

Enter command name: ls

Executable Found:
/usr/bin/ls
root@SRIVALLI:~# abcxyz
abcxyz: command not found
root@SRIVALLI:~# echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/
/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/Common Fil
javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/
c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/
m32:/mnt/c/WINDOWS:/mnt/c/WINDOWS/System32/Wbem:/mnt/c/WINDOW
owsPowerShell/v1.0:/mnt/c/WINDOWS/System32/OpenSSH:/mnt/c/P
tnet:/mnt/c/Program Files/nodejs:/mnt/c/Users/sriva_6wg39hp
Microsoft/WindowsApps:/mnt/c/Users/sriva_6wg39hp/AppData/Loca
o soft VS Code/bin:/mnt/c/Program Files/JetBrains/PyCharm 2026
sers/sriva_6wg39hp/AppData/Roaming/npm:/mnt/c/Users/sriva_6wg
cal/Kingsoft/WPS Office/12.1.0.27458/clitool:/snap/bin
root@SRIVALLI:~# which ls
/usr/bin/ls
root@SRIVALLI:~# ls -l /bin/ls
-rwxr-xr-x 1 root root 142312 Jan 23  2026 /bin/ls
root@SRIVALLI:~# which gcc
which pwd
which date
/usr/bin/gcc
/usr/bin/pwd
/usr/bin/date
root@SRIVALLI:~# |

```

Report:

In this task, the PATH environment variable was retrieved and parsed to identify the directories used for command searching. The program searched these directories to locate executable files and verified their execution permissions. Valid and invalid commands were tested to understand command resolution and error handling in Linux. The PATH variable was retrieved successfully, executable files were located correctly, and missing commands were handled appropriately during command resolution.

Task-3: To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    char *user = getenv("USER");
    char *home = getenv("HOME");
    char *shell = getenv("SHELL");
    char *test = getenv("TEST");

    printf("Variable Expansion:\n");

    if(user)
        printf("$USER = %s\n", user);

    if(home)
        printf("$HOME = %s\n", home);

    if(shell)
        printf("$SHELL = %s\n", shell);

    if(test)
        printf("$TEST = %s\n", test);
    else
        printf("$TEST is undefined\n");

    return 0;
}
```

Result:

```

root@SRIVALLI:~# gcc variable_expansion.c -o variable_expansion
root@SRIVALLI:~# ./variable_expansion
Variable Expansion:
$USER = root
$HOME = /root
$SHELL = /bin/bash
$TEST is undefined
root@SRIVALLI:~# echo $USER
root
root@SRIVALLI:~# echo $HOME
/root
root@SRIVALLI:~# echo $SHELL
/bin/bash
root@SRIVALLI:~# export TEST="LinuxLab"
root@SRIVALLI:~# echo $TEST
LinuxLab
root@SRIVALLI:~# echo $TEST
LinuxLab
root@SRIVALLI:~# ./variable_expansion
Variable Expansion:
$USER = root
$HOME = /root
$SHELL = /bin/bash
$TEST = LinuxLab
root@SRIVALLI:~# echo $ABC

root@SRIVALLI:~# export NAME="Srivalli"
export MSG="Hello"
echo $MSG $NAME
Hello Srivalli
root@SRIVALLI:~# env
SHELL=/bin/bash
WSL2_GUI_APPS_ENABLED=1
WSL_DISTRO_NAME=Ubuntu
MSG=Hello
NAME=Srivalli
PWD=/root
LOGNAME=root
HOME=/root
LANG=C.UTF-8
WSL_INTEROP=/run/WSL/729_interop

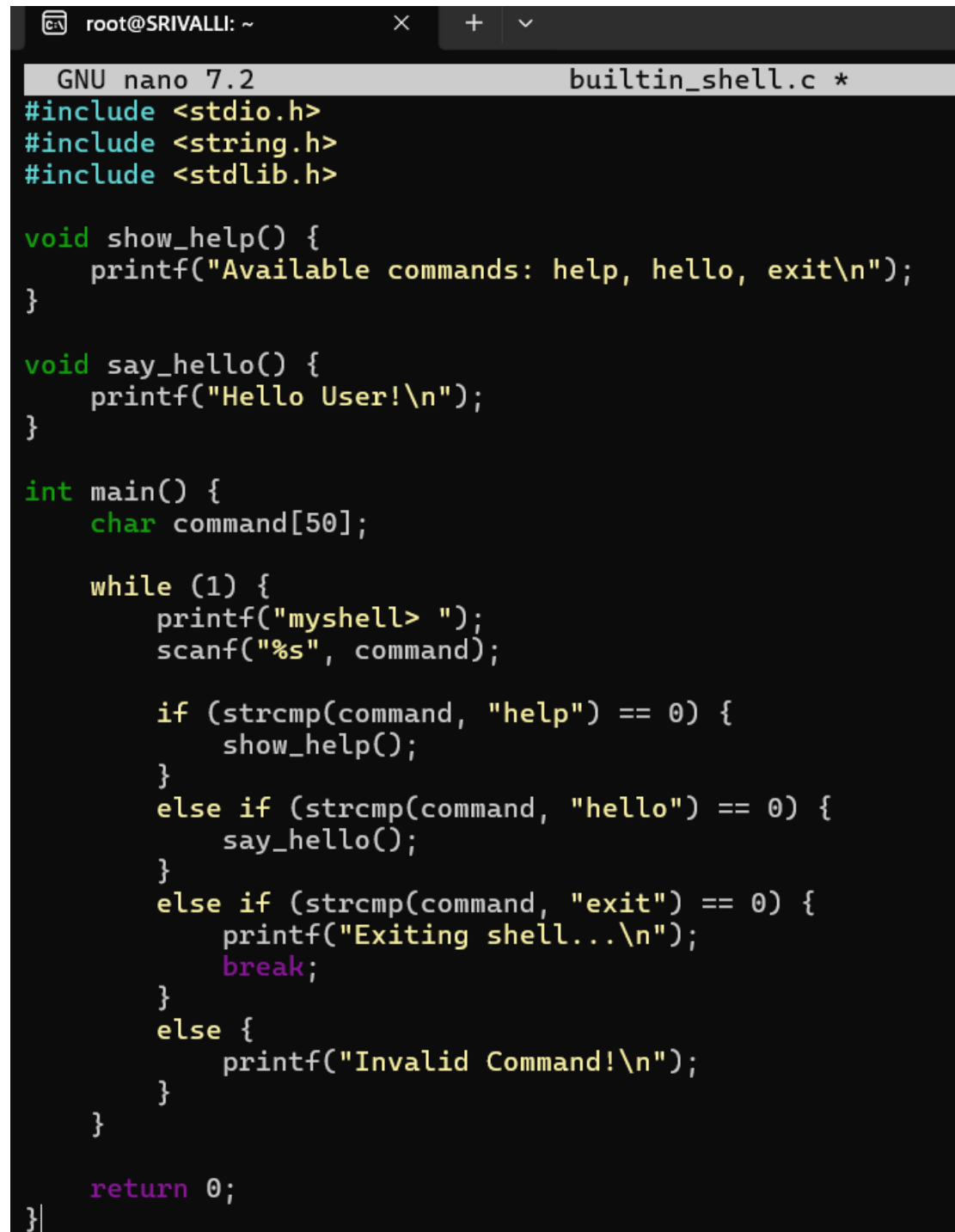
```

Report:

In this task, environment variable references were detected and their values were expanded during execution. Both defined and undefined variables were tested to observe system behavior. Variable values were updated dynamically, and nested variable usage was verified using different test cases. This helped in understanding how shells perform variable expansion before command execution. Variable references were detected successfully, values

were expanded correctly, undefined variables were handled properly, and variable expansion logic was verified through multiple test cases.

Task-4: To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic



```
root@SRIVALLI: ~
GNU nano 7.2 builtin_shell.c *
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void show_help() {
    printf("Available commands: help, hello, exit\n");
}

void say_hello() {
    printf("Hello User!\n");
}

int main() {
    char command[50];

    while (1) {
        printf("myshell> ");
        scanf("%s", command);

        if (strcmp(command, "help") == 0) {
            show_help();
        }
        else if (strcmp(command, "hello") == 0) {
            say_hello();
        }
        else if (strcmp(command, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }
        else {
            printf("Invalid Command!\n");
        }
    }

    return 0;
}
```

Result:

```

HOSTTYPE=x86_64
PULSE_SERVER=unix:/mnt/wslg/PulseServer
TEST=LinuxLab
_=/usr/bin/env
root@SRIVALLI:~# printenv HOME
/root
root@SRIVALLI:~# unset TEST
root@SRIVALLI:~# echo $TEST

root@SRIVALLI:~# nano builtin_shell.c
root@SRIVALLI:~# gcc builtin_shell.c -o builtin_shell
root@SRIVALLI:~# ./builtin_shell
myshell> myshell> help
Invalid Command!
myshell> Available commands: help, hello, exit
myshell> myshell> he^Z
[1]+  Stopped                  ./builtin_shell
root@SRIVALLI:~#
root@SRIVALLI:~# myshell> abc
Command 'myshell' not found, did you mean:
  command 'pyshell' from deb wxpython-tools (4.2.1+dfsg-3)
Try: apt install <deb name>
root@SRIVALLI:~# ./builtin_shell
myshell> help
Available commands: help, hello, exit
myshell> hello
Hello User!
myshell> cat builtin_shell.c

```

Report:

In this task, built-in commands were identified and executed within the shell process. A simple dispatch mechanism was implemented to route commands to the appropriate functions. Invalid commands were handled with error messages, and the shell maintained its state while processing multiple commands. Different inputs were tested to verify the command dispatch logic. Built-in commands were executed successfully, invalid commands were detected correctly, and the dispatch logic worked as expected.